📘 MODULE 8: EVAL

8.1 Concept

eval **creates or changes a field** by calculating an expression, on each event.

8.2 Why it matters

Used to label risk, calculate durations, clean values, and build conditions for alerts and dashboards.

8.3 Syntax

spl

| eval <new_field> = <expression>

Multiple assignments: | eval a=1, b=a+1 (evaluated left to right).

8.4 Rules

- Strings go in **double quotes**: "high".
- Field names are bare: count. If the field name contains special characters, use **single quotes**: 'my-field'.
- **== compares, = assigns.** Inside if(), case(), and where, use ==.
- Concatenate strings with the **dot** operator: first." ".last.
- Field names are **case-sensitive**.

8.5 Operations and functions

Type	Examples
Math	+ - * / %, round(x,2), abs()
String	lower(), upper(), len(), substr(x,1,3), replace(x,"a","b"), trim()
Conditional	if(cond, then, else), case(c1,v1, c2,v2, true(),default)
Pattern	match(x,"regex"), like(x,"adm%") (uses % as wildcard)
Null handling	isnull(x), isnotnull(x), coalesce(a,b), null()
Convert	tostring(), tonumber()
Time	strftime(_time,"%F %T"), strptime(...)
Multivalue	mvcount(), mvindex() (Module 14)

8.6 Worked queries

Query 1: static field

spl

index=_internal sourcetype=splunkd | eval risk="high" | table _time component risk

What it does: adds a risk field to every event with value high.

Query 2: math

spl

| makeresults | eval a=10, b=5, total=a+b, ratio=round(a/b,2)

What it does: makeresults generates one empty event with the current time. It's a free testing sandbox. Result: total=15, ratio=2.00.

Query 3: if()

spl

index=_internal sourcetype=splunkd_access

| stats count by clientip

| eval status=if(count>10,"High","Low")

What it does: after stats, each IP row gets High if its count exceeds 10, else Low.
Line-by-line: if(condition, value_if_true, value_if_false).
Common mistake: if(count=10,...) should be ==.

Query 4: case() for multiple tiers

spl

index=_internal sourcetype=splunkd_access

| eval category=case(status>=500,"Server Error", status>=400,"Client Error", status>=300,"Redirect", status>=200,"Success", true(),"Other")

| stats count by category

What it does: classifies HTTP status codes. case returns the value of the **first true condition**, so true() is the catch-all default. Without it, unmatched events get null.

Query 5: strings, match, like

spl

| makeresults

| eval user="Admin_John", u=lower(user), is_admin=if(like(u,"admin%"),"yes","no"), has_digits=if(match(user,"\d"),"yes","no"), label=user."-"."prod"

Result: u=admin_john, is_admin=yes, has_digits=no, label=Admin_John-prod.
Common mistake: in like, use % not *.

Query 6: nulls

spl

... | eval user=coalesce(user, src_user, "unknown")

... | fillnull value="N/A" user

coalesce returns the first **non-null** argument. fillnull (Module 14) replaces nulls across fields.

8.7 SOC example: label a failed login volume

spl

index=* sourcetype=linux_secure "Failed password"

| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"

| stats count by src_ip

| eval severity=case(count>=100,"critical", count>=20,"high", count>=5,"medium", true(),"low")

| sort - count

Interview explanation: "I extract the source IP, count failures per IP, and use case to map counts to severity tiers. In production, these thresholds would be tuned."

8.8 Hands-on

1. Using makeresults, create a field price=250 and tax=price*0.18, then total.
2. Take splunkd events and create is_error = "yes" if log_level equals ERROR, otherwise "no". Then stats count by is_error.
3. Convert bytes in splunkd_access to KB with 2 decimals.
4. Write a case() that labels status as Success/Redirect/Error.

8.9 Troubleshooting

Symptom	Cause
New field is empty	Source field name typo or case mismatch

Symptom	Cause
if always false	Used = instead of ==, or compared string without quotes
eval result missing after stats	stats dropped the field, so run eval after it or include it in stats
Math returns null	A field is null or non-numeric, so use tonumber() or coalesce

8.10 Interview Q&A

- **What does eval do?** Creates or modifies a field using an expression.
- **if vs case?** if handles two outcomes, case handles many (first match wins).
- **eval vs stats?** eval works per event, stats aggregates across events.
- **How do you handle null?** isnull, coalesce, fillnull.

8.11 Revision

= assigns, == compares · strings in "..." · concatenate with . · case ends with true() · like uses %.